

Partie 1: Intro

Bonjour à tous,

Aujourd'hui, nous venons vous présenter l'avancement de notre projet.

Comme vous le savez, l'objectif est de modéliser un robot sous MATLAB avec la Robotics System Toolbox, de valider ce modèle sur des tâches de suivi de trajectoire, puis d'aller progressivement vers une commande en contact.

Pour ce projet, nous avons choisi de travailler sur le UR5 de Universal Robots. Il s'agit d'un cobot, donc d'un robot collaboratif, très utilisé en industrie pour des tâches comme le pick and place, l'assemblage ou la manipulation d'objets. Ce choix nous a semblé pertinent parce qu'il s'agit d'un robot à la fois réaliste, largement utilisé, et suffisamment complet pour couvrir les différentes parties du projet. Nous l'avons également retenu car c'est un robot bien documenté, notamment pour tout ce qui concerne les données nécessaires à la modélisation dynamique, ce qui sera important pour la suite.

Avant de nous lancer directement sur l'UR5, nous avons d'abord pris en main MATLAB et la toolbox à travers de petits exercices. Nous avons commencé par créer un robot 2R avec des objets de type rigidBodyTree, afin de comprendre la structure générale de modélisation d'un robot dans MATLAB, ainsi que la gestion des articulations, des liens et des transformations. Nous avons ensuite testé plusieurs configurations pour vérifier que le comportement obtenu était cohérent. Dans un second temps, nous avons utilisé ce même robot pour réaliser un premier test de cinématique inverse, afin de relier une position souhaitée de l'effecteur aux variables articulaires. Cette étape nous a servi de base avant de passer à un robot plus complexe.

Partie 2 : cinématique directe

Nous avons ensuite travaillé sur la modélisation cinématique directe de l'UR5 en utilisant la convention de Craig, donc le DH modifié vu dans le cours. La difficulté ici, c'est que Universal Robots fournit principalement les paramètres en DH standard, et non directement dans la convention modifiée que nous utilisons. Pour valider notre modèle, nous sommes donc partis d'une idée simple : même si la convention change, le résultat géométrique final doit rester le même. Autrement dit, pour une même configuration physique du robot, on doit retrouver la même pose de l'effecteur.

Pour cela, nous nous sommes appuyés sur un tableur fourni par **Universal Robots**, qui permet de calculer la pose du robot à partir des paramètres constructeur et d'une configuration articulaire donnée. De notre côté, nous avons implémenté sous MATLAB notre propre modèle de **cinématique directe** à partir du DH

modifié. En injectant une même pose test dans les deux approches, nous obtenons la même position et la même orientation de l'effecteur, ce qui nous a permis de valider notre modèle direct.

Nous avons ensuite complété cette validation par une deuxième méthode. Nous sommes partis d'une **pose réelle du robot**, récupérée depuis l'interface d'Universal Robots, en choisissant volontairement une configuration non singulière. Nous avons relevé les **angles articulaires**, puis nous les avons injectés dans notre modèle direct pour calculer la pose de l'effecteur. Ensuite, nous avons pris cette pose cible et nous l'avons utilisée dans MATLAB avec le modèle **URDF de l'UR5**, chargé via la Robotics System Toolbox. Nous avons alors utilisé l'outil **inverse kinematics** pour retrouver les variables articulaires correspondantes. L'idée était donc de faire la vérification dans les deux sens : partir des articulations pour obtenir la pose, puis repartir de la pose pour retrouver les articulations. Les résultats obtenus étant cohérents, cela nous a permis de confirmer une seconde fois la validité de notre modèle cinématique direct.

Partie 3 : cinématique inverse

Après cela, nous avons utilisé la **cinématique inverse** dans le cadre d'un **suivi de trajectoire**. Cette fois, l'objectif n'était plus seulement de valider une pose isolée, mais de vérifier que notre modèle permet de faire évoluer l'effecteur le long d'une trajectoire imposée. Pour cela, nous sommes partis d'une pose initiale déjà validée, puis nous avons défini une **trajectoire cartésienne simple**, avec un déplacement rectiligne selon l'axe **X**, tout en gardant une orientation constante. Pour chacun des points de cette trajectoire, nous avons utilisé l'outil **inverse kinematics** de MATLAB afin de calculer les angles articulaires correspondants. La solution calculée à un instant sert ensuite de point de départ pour le calcul du point suivant, ce qui permet d'obtenir une trajectoire articulaire continue et cohérente. Cette étape constitue une première validation fonctionnelle du projet, puisqu'on ne valide plus seulement une pose, mais bien une **suite de poses**.

Partie 4: cinématique différentielle

Nous avons ensuite abordé la **cinématique différentielle** à travers le calcul de la **jacobienne**. Sur le plan théorique, nous sommes repartis de la position de l'effecteur donnée par notre modèle direct, puis nous avons calculé les **dérivées partielles** de cette position par rapport aux variables articulaires. Cela permet d'obtenir la jacobienne cinématique, qui relie les vitesses articulaires aux vitesses de l'effecteur. En parallèle, sous MATLAB, nous avons repris cette partie avec la toolbox en nous appuyant sur la fonction **geometricJacobian**, qui permet d'obtenir la jacobienne géométrique du modèle du robot pour une configuration donnée. Cela nous donne une

référence directe côté MATLAB pour la suite de l'étude.

Partie 5 : dynamique

Enfin, nous avons commencé la partie dynamique du projet. L'idée, ici, est de passer d'un modèle purement géométrique, qui décrit uniquement la position et le mouvement du robot, à un modèle capable de relier ce mouvement aux couples appliqués dans les articulations. En théorie, cela se traduit par l'équation dynamique classique du robot, dans laquelle on retrouve la matrice d'inertie, les termes liés aux vitesses — donc les effets centrifuges et de Coriolis — ainsi que le terme de gravité. L'objectif est donc de pouvoir déterminer les efforts nécessaires pour produire un mouvement donné. Sous MATLAB, nous avons réalisé cette implémentation à partir du modèle UR5 chargé dans la Robotics System Toolbox. La toolbox met à disposition plusieurs fonctions utiles, comme `massMatrix`, `velocityProduct`, `gravityTorque` et `inverseDynamics`, qui permettent d'accéder aux différentes composantes du modèle dynamique du robot. Cette étape est importante, car elle prépare directement la suite du projet, notamment pour la mise en place de la commande en contact.

conclusion :

La prochaine étape sera maintenant de consolider cette partie dynamique, puis de passer à la **commande en contact**, soit en **impédance**, soit en **commande hybride force-position**.